

Relatório de Testes e Qualidade - Fase 02

1. Lista de Testes Implementados.

Teste	Arquivo	O que valida	Status
testVantagemElemental()	pokesal/PokesalTest.java	Valida os multiplicadores de dano por tipo elemental: Fogo x2.0 em Planta, Água x2.0 em Fogo, Planta x2.0 em Água (e x0.5 no sentido inverso; x1.0 entre tipos iguais).	Falhou (esperado 0.5, retornou 1.0 em Fogo vs Água)
testTerrenoDoEstacionamento()	pokesal/PokesalTest.java	Valida os bônus de terreno: Asfalto Quente (+1.5 de dano de Fogo) e Poça de Chuva (+1 de dano de Água), usando Pokésal do mesmo tipo pra isolar o efeito do terreno.	Falhou (esperado 11.0, retornou 10.0 na Poça de Chuva)
testQuemAtacaPrimeiro()	pokesal/PokesalTest.java	Confirma que quem tem mais velocidade (SPD) ataca primeiro, e que entrar em estado crítico aumenta a SPD em 1.25x.	Passou
testLimitesDeHp()	pokesal/PokesalTest.java	Valores de fronteira (boundary values): HP não passa do máximo curando, não fica negativo tomando dano, e cura/dano extremos ficam travados em 100/0.	Passou
testLimiteDeItensNaBatalha()	pokesal/TreinadorTest.java	Garante que depois de usar os 2 itens permitidos por batalha, podeUsarItem() passa a barrar um 3º uso.	Passou
testEstadoCritico()	pokesal/PokesalRequisitosAutoraisTest.java	Requisito autoral 1: com HP abaixo de 30, o Pokésal entra em estado crítico e tanto o dano quanto a SPD aumentam em 1.25x.	Passou
testAcertoCritico()	pokesal/PokesalRequisitosAutoraisTest.java	Requisito autoral 2: acerto crítico soma +5 de dano fixo (forçado via parâmetro no teste, sem depender do sorteio de 10% da Batalha).	Passou

2. Checklist de Teste Estático de Código.

Pergunta	Resposta	Onde encontrado / Observação
Todas as variáveis estão inicializadas antes do uso?	Sim	Todos os atributos são inicializados no construtor ou na declaração; valores fixos foram extraídos para constantes (ex: HP CRITICO, BONUS CRITICO).
Há variáveis declaradas e nunca usadas?	Não	Não foi encontrado nenhum caso nas classes analisadas.
Existe código inacessível?	Não	Todos os fluxos são alcançáveis.
Há código duplicado?	Sim	Treinador.usarPotion() e usarSuperPotion() têm lógica quase idêntica, diferindo só na constante de cura e no texto da mensagem.
Existem erros de sintaxe?	Não	Código compila normalmente.
Há erros de lógica que quebram regras do negócio?	Sim (2 pontos)	(1) O bônus de estado crítico e a penalidade de queimado em calcularDano() usam o estado do atacante, não do defensor. (2) O documento define Acerto Crítico como dano x1,5, mas o código aplica soma fixa de 5 (BONUS_ACERTO_CRITICO), não uma multiplicação.
Há erros de tipagem?	Não	Tipos usados corretamente (String para tipo/status/terreno, int para HP/ATK/SPD).
O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?	Sim	Os laços while dependem de entrada validada por Entrada.lerInteiro() ou de condição de fim de batalha, sem risco identificado de loop infinito.
Outros tipos de erros identificados?	Sim	tipo, status e terreno continuam representados como String livre (não enum), permitindo digitar um valor incorreto sem erro de compilação, mas quebrando silenciosamente a comparação .equals(...).

3. Matriz de Rastreabilidade.

Liga cada requisito de negócio ao teste que o cobre

Requisito	Teste que valida
Vantagem elemental Fogo/Água/Planta	testVantagemElemental()
Efeito do terreno (Estacionamento da UCSal)	testTerrenoDoEstacionamento()
Ordem de ataque por velocidade (SPD)	testQuemAtacaPrimeiro()
Limites de HP (valores limites)	testLimitesDeHp()
Limite de 2 itens por batalha	testLimiteDeItensNaBatalha()
Requisito autoral 1 (estado crítico)	testEstadoCritico()
Requisito autoral 2 (acerto crítico)	testAcertoCritico()

4. Análise SonarQube.

Métrica	Resultado
Bugs	0 nenhuma das 66 ocorrências do SonarLint corresponde a uma regra do tipo Bug (todas são Code Smells de estilo/manutenibilidade)
Vulnerabilities	0 nenhuma ocorrência relacionada a segurança foi identificada entre as 66 regras levantadas
Code Smells	66
Coverage (%)	34.6%

Print do relatório do SonarLint no Eclipse (66 itens encontrados, todos Code Smells)

5. Bugs Encontrados e Correções

Bug encontrado	Onde foi identificado	Correção realizada
Ordem de ataque comparava o nome do treinador em vez do nome do Pokésal	Teste JUnit (testOrdemDeAtaquePorVelocidade)	Teste ajustado para verificar rapido.getPokesal().getNome() em vez de rapido.getNome()
Bônus de terreno "Poça de Chuva" não é aplicado ao dano de Água (esperado 11.0, retornou 10.0)	Teste JUnit (testTerrenoDoEstacionamento) — falha confirmada na execução	Pendente — revisar cálculo de dano para o terreno Poça de Chuva
Multiplicador de tipo Fogo vs Água retorna 1.0 (neutro) em vez de 0.5 (fraqueza)	Teste JUnit (testVantagemElemental, PokesalTest.java:18) — falha confirmada na execução	Pendente — revisar método multiplicadorDeTipo() para o caso Fogo vs Água

Cobertura de Testes com JUnit:

```
package pokesal;
```

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;
```

```
class PokesalTest {
```

```
    // testando se a vantagem de tipo ta funcionando certo (fogo > planta, agua > fogo, planta > agua)
```

```
    @Test
```

```
    void testVantagemElemental() {
```

```
        Pokesal charSal = new Pokesal("CharSal", "Fogo", 70);
```

```
        Pokesal bulbaSal = new Pokesal("BulbaSal", "Planta", 50);
```

```
        Pokesal squirtSal = new Pokesal("SquirtSal", "Agua", 40);
```

```
        // fogo bate forte em planta
```

```
        assertEquals(2.0, charSal.multiplicadorDeTipo(bulbaSal));
```

```
        // mas fogo apanha de agua
```

```
        assertEquals(0.5, charSal.multiplicadorDeTipo(squirtSal));
```

```
        assertEquals(2.0, squirtSal.multiplicadorDeTipo(charSal);
```

```
        assertEquals(0.5, squirtSal.multiplicadorDeTipo(bulbaSal));
```

```
        assertEquals(2.0, bulbaSal.multiplicadorDeTipo(squirtSal));
```

```
        assertEquals(0.5, bulbaSal.multiplicadorDeTipo(charSal));
```

```
        // dois do mesmo tipo nao deve dar bonus nenhum
```

```
        Pokesal outroFogo = new Pokesal("CyndaSal", "Fogo", 80);
```

```
        assertEquals(1.0, charSal.multiplicadorDeTipo(outroFogo));
```

```
    }
```

```

@Test
void testTerrenoDoEstacionamento() {
    Pokesal charSal = new Pokesal("CharSal", "Fogo", 70);
    Pokesal cyndaSal = new Pokesal("CyndaSal", "Fogo", 80); // usei mesmo tipo pra nao ter vantagem elemental atrapalhando
    Pokesal squirtSal = new Pokesal("SquirtSal", "Agua", 40);
    Pokesal totoSal = new Pokesal("TotoSal", "Agua", 50);

    // sem terreno nenhum, o dano do CharSal deve ser só o ataque base (10)
    assertEquals(10.0, charSal.calcularDano(cyndaSal, "Nenhum", false));

    // no asfalto quente, fogo ganha +1.5 de dano
    assertEquals(11.5, charSal.calcularDano(cyndaSal, "Asfalto Quente", false));

    // na poça de chuva, agua ganha +1 de dano
    assertEquals(10.0, squirtSal.calcularDano(totoSal, "Nenhum", false));
    assertEquals(11.0, squirtSal.calcularDano(totoSal, "Poça de Chuva", false));
}

```

```

@Test
void testQuemAtacaPrimeiro() {
    // quem tem mais velocidade ataca primeiro, entao a spd efetiva do mais rapido tem que ser maior
    Pokesal rapido = new Pokesal("CyndaSal", "Fogo", 80);
    Pokesal lento = new Pokesal("SquirtSal", "Agua", 40);

    assertTrue(rapido.getSpdEfetiva() > lento.getSpdEfetiva());

    // e se ele entrar em estado critico a velocidade dele aumenta ainda mais (1.25x)
    rapido.receberDano(75); // deixa ele com 25 de hp, abaixo de 30
    assertEquals(100.0, rapido.getSpdEfetiva()); // 80 * 1.25
}

```

```

@Test
void testLimitesDeHp() {
    Pokesal pokesal = new Pokesal("TotoSal", "Agua", 50);

    // nao pode passar do hp maximo mesmo curando muito
    pokesal.setHp(150);
    assertEquals(100, pokesal.getHp());

    // nao pode ficar negativo tambem
    pokesal.setHp(-20);
    assertEquals(0, pokesal.getHp());
    assertTrue(pokesal.estaDerrotado());

    // curando de 0 hp nao pode passar do maximo
    pokesal.curar(200);
    assertEquals(100, pokesal.getHp());
}

```

```
// um dano gigante nao pode deixar o hp negativo, tem que travar em 0
pokesal.receberDano(1000);
assertEquals(0, pokesal.getHp());
}
```

```
class TreinadorTest {
```

```
    @Test
```

```
    void testLimiteDeItensNaBatalha() {
```

```
        Treinador treinador = new Treinador("Ash", new Pokesal("CharSal", "Fogo", 70));
```

```
        // no comeco ele pode usar item numa boa
```

```
        assertTrue(treinador.podeUsarItem());
```

```
        treinador.usarPotion();
```

```
        assertTrue(treinador.podeUsarItem()); // ainda pode, ja que o limite é 2
```

```
        treinador.usarSuperPotion();
```

```
        assertEquals(2, treinador.getItensUsados());
```

```
        // depois de usar os 2 itens permitidos, nao pode mais
```

```
        assertFalse(treinador.podeUsarItem());
```

```
    }
```

```
class PokesalRequisitosAutoraisTest {
```

```
    @Test
```

```
    void testEstadoCritico() {
```

```
        // pokesal do mesmo tipo pra nao entrar vantagem elemental no meio do calculo
```

```
        Pokesal atacante = new Pokesal("CharSal", "Fogo", 70);
```

```
        Pokesal alvo = new Pokesal("CyndaSal", "Fogo", 80);
```

```
        double danoAntes = atacante.calcularDano(alvo, "Nenhum", false);
```

```
        assertEquals(10.0, danoAntes);
```

```
assertEquals(70.0, atacante.getSpdEfetiva());
```

```
// tira quase toda a vida do atacante pra ele entrar em estado critico (hp < 30)
```

```
atacante.receberDano(75);
```

```
assertTrue(atacante.estaCritico());
```

```
// agora o dano e a velocidade deveriam ter aumentado 1.25x
```

```
double danoDepois = atacante.calcularDano(alvo, "Nenhum", false);
```

```
assertEquals(12.5, danoDepois);
```

```
assertEquals(87.5, atacante.getSpdEfetiva());
```

```
}
```

```
@Test
```

```
void testAcertoCritico() {
```

```
    Pokesal atacante = new Pokesal("CharSal", "Fogo", 70);
```

```
    Pokesal alvo = new Pokesal("CyndaSal", "Fogo", 80);
```

```
// o metodo calcularDano ja recebe se foi critico ou nao, entao da pra forçar isso no teste
```

```
// sem depender do sorteio de 10% que rola la na Batalha
```

```
double semCritico = atacante.calcularDano(alvo, "Nenhum", false);
```

```
double comCritico = atacante.calcularDano(alvo, "Nenhum", true);
```

```
assertEquals(10.0, semCritico);
```

```
assertEquals(15.0, comCritico); // no codigo o critico soma +5 no dano
```

```
assertTrue(comCritico > semCritico);
```

```
}
```

```
}
```